

Database Management System - 2

(Relational)

AUTOCOMMIT, COMMIT, and ROLLBACK in MySQL

Transactions are an important concept in database management systems. They ensure that a group of SQL operations is executed safely and consistently. MySQL manages transactions using statements such as **AUTOCOMMIT**, **COMMIT**, and **ROLLBACK**. These commands help maintain **data integrity**, especially when multiple related operations must either succeed together or fail together.

This is particularly important in systems like banking applications, inventory management systems, or data science workflows where large data updates and complex operations are common.

What is a Transaction

A **transaction** is a sequence of one or more SQL statements that are executed as a **single unit of work**. The database system ensures that either all operations in the transaction are completed successfully or none of them are applied.

Transactions follow four important properties known as **ACID properties**, which guarantee reliability and consistency in the database.

ACID Properties of Transactions

Atomicity

Atomicity means **all operations in a transaction are completed entirely or not at all**. If any operation fails during the transaction, the entire transaction is canceled and the database returns to its previous state.

Example

Consider a bank transfer where money is deducted from one account and added to another.

START TRANSACTION;

UPDATE accounts SET balance = balance - 100 WHERE id = 1;

UPDATE accounts SET balance = balance + 100 WHERE id = 2;

COMMIT;

If the second update fails, the first update will also be undone to maintain consistency.

Consistency

Consistency ensures that the database moves from **one valid state to another valid state** after a transaction is completed. All database rules, constraints, and relationships must remain valid.

Example

If a database rule prevents negative balances, any transaction that attempts to create a negative balance will fail.

Isolation

Isolation ensures that **multiple transactions running at the same time do not interfere with each other**. Each transaction works independently until it is completed.

Example

If two users are updating the same record simultaneously, the database ensures that each transaction operates separately without seeing incomplete changes from the other.

Durability

Durability guarantees that **once a transaction is committed, the changes are permanently stored in the database**. Even if the system crashes afterward, the committed changes will remain saved.

Example

If a transaction updates customer information and commits successfully, the changes will still exist even if the server restarts.

AUTOCOMMIT in MySQL

By default, MySQL runs in **autocommit mode**. In this mode, every SQL statement is treated as a separate transaction and is automatically committed immediately after execution.

This means that any INSERT, UPDATE, or DELETE statement will automatically save changes to the database without requiring a manual commit.

This behavior is convenient for simple operations but may not be suitable when multiple related operations must be executed together.

Checking Autocommit Status

You can check whether autocommit is enabled or disabled using the following query:

```
SELECT @@autocommit;
```

Possible results:

- **1** → Autocommit is enabled
- **0** → Autocommit is disabled

Disabling Autocommit

When autocommit is disabled, multiple SQL statements can be grouped into a single transaction. The changes will only be saved when a COMMIT statement is executed.

```
SET autocommit = 0;
```

This is useful when performing multiple dependent operations.

Enabling Autocommit

Autocommit mode can be enabled again if needed.

```
SET autocommit = 1;
```

COMMIT Statement

The **COMMIT** statement permanently saves all the changes made during the current transaction. Once a transaction is committed:

- The changes become visible to other users
- The changes are permanently stored in the database

Example of COMMIT

```
START TRANSACTION;
```

```
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
```

```
UPDATE accounts SET balance = balance + 100 WHERE id = 2;
```

```
COMMIT;
```

Explanation

- The transaction begins with START TRANSACTION
- Money is deducted from account **1**
- The same amount is added to account **2**
- COMMIT saves both operations permanently

This ensures the transfer happens successfully as a single operation.

ROLLBACK Statement

The **ROLLBACK** statement is used to **undo all changes made during the current transaction**. It is useful when an error occurs or when certain conditions are not satisfied.

When a rollback occurs, the database returns to the state it was in before the transaction started.

Example of ROLLBACK

START TRANSACTION;

UPDATE accounts SET balance = balance - 100 WHERE id = 1;

-- Suppose an error occurs here

ROLLBACK;

Explanation

- The transaction starts
- The account balance is updated
- An error occurs before the transaction completes
- ROLLBACK cancels the transaction and restores the original data

All changes made since the transaction started are discarded.

When working with transactions, the following practices help maintain database reliability.

- Always use transactions when performing multiple related database operations
- Disable autocommit when working with critical or sensitive data updates
- Use ROLLBACK immediately if any step in the transaction fails
- Perform validation checks before committing a transaction
- Thoroughly test transactions before executing them on production data

Getting Current Date and Time in MySQL

MySQL provides several built-in functions that return the **current date, time, or timestamp**. These functions are commonly used in logging systems, event tracking, auditing, and time-based queries.

They are also widely used in data science workflows to track when records are created or updated.

CURRENT_DATE

CURRENT_DATE returns the **current date only** in the format **YYYY-MM-DD**.

Example

SELECT CURRENT_DATE;

Example Output

2001-01-01

Use cases include recording the date of attendance, order date, or any event where time is not required.

Example

```
INSERT INTO attendance (employee_id, date)
VALUES (101, CURRENT_DATE);
```

CURRENT_TIME

CURRENT_TIME returns the **current time only** in the format **HH:MM:SS**.

Example

```
SELECT CURRENT_TIME;
```

Example Output

09:19:34

This function is useful for recording time-based logs or system activities where only time information is required.

CURRENT_TIMESTAMP and NOW()

CURRENT_TIMESTAMP returns both the **current date and time**. The function NOW() performs the same task and is widely used in MySQL queries.

Example

```
SELECT CURRENT_TIMESTAMP;
```

or

```
SELECT NOW();
```

Example Output

2001-01-01 09:29:19

These functions are commonly used for storing the creation time or update time of records.

Example

```
INSERT INTO logs (event, created_at)
VALUES ('data_import', NOW());
```

This automatically records the exact date and time when the event occurred.

LOCALTIME and LOCALTIMESTAMP

LOCALTIME and LOCALTIMESTAMP are **synonyms for NOW()** and return the current date and time.

Example

```
SELECT LOCALTIME;
```

```
SELECT LOCALTIMESTAMP;
```

Example Output

2001-01-01 09:29:19

Important Clarification About Local Time

The word “**local**” in LOCALTIME refers to the **time zone configured on the MySQL server**, not the user's system time.

This means the returned value is based on the server's configured time zone rather than the client computer's time.

Example

If the MySQL server is configured to use **UTC** but the user is in **India**, LOCALTIME will still return the server's UTC time.

Using Date and Time Functions in Table Inserts

Date and time functions can be used while inserting records so that timestamps are automatically recorded.

Example

```
INSERT INTO logs (event, created_at)
```

```
VALUES ('data_import', NOW());
```

This automatically stores the exact time when the event was recorded.

Example

```
CREATE TABLE logs (
```

```
  id INT AUTO_INCREMENT PRIMARY KEY,
```

```
  event VARCHAR(100),
```

```
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

);

In this table, the created_at column automatically stores the current date and time whenever a new record is inserted.

Constraints in SQL

Constraints in SQL are rules applied to table columns to enforce **data integrity, consistency, and validity**. They restrict the type of data that can be inserted into a table and help ensure that the stored data follows predefined rules.

By applying constraints, the database system automatically checks data before allowing it to be inserted or updated. This prevents invalid, duplicate, or incomplete data from entering the database.

Constraints are an essential part of database design because they help maintain **accurate and reliable data** without relying only on application-level validation.

Why Use Constraints

Constraints play a critical role in maintaining the quality and reliability of data stored in databases.

Key reasons for using constraints include:

- Ensure **data quality and accuracy** in tables
- Prevent **invalid or incorrect data** from being stored
- Avoid **duplicate entries** where uniqueness is required
- Ensure that required fields are always filled
- Enforce **business rules directly at the database level**
- Improve overall **data consistency and reliability**

NOT NULL Constraint

The **NOT NULL** constraint ensures that a column **cannot contain NULL values**. This means that every row must have a valid value for that column when a record is inserted.

This constraint is commonly used for fields that must always contain data.

Typical use cases include identifiers, names, or contact information that should never be left empty.

Example

```
CREATE TABLE employees (  
    id INT NOT NULL,  
    name VARCHAR(100) NOT NULL  
);
```

Explanation

- The id column must always contain a value
- The name column must also contain a value
- If an insert operation does not provide values for these columns, the database will return an error

Example Insert

Valid insert

```
INSERT INTO employees (id, name)  
VALUES (101, 'Rahul');
```

Invalid insert

```
INSERT INTO employees (id)  
VALUES (102);
```

This will fail because the name column cannot be NULL.

UNIQUE Constraint

The **UNIQUE** constraint ensures that all values in a column are **distinct**. Duplicate values are not allowed in columns with this constraint.

This is commonly used for attributes that must be unique across records, such as usernames, email addresses, or identification numbers.

Example

```
CREATE TABLE users (  
    username VARCHAR(50) UNIQUE,  
    email VARCHAR(100) UNIQUE  
);
```

Explanation

- Each username must be unique
- Each email must also be unique
- Attempting to insert duplicate values will result in an error

Example

Valid inserts

```
INSERT INTO users VALUES ('john123', 'john@email.com');  
INSERT INTO users VALUES ('alex45', 'alex@email.com');
```

Invalid insert

```
INSERT INTO users VALUES ('john123', 'another@email.com');
```

This will fail because john123 already exists.

Important Note

- A table can have **multiple UNIQUE constraints**
- However, a table can have **only one PRIMARY KEY**

DEFAULT Constraint

The **DEFAULT** constraint assigns a **default value** to a column when no value is provided during an insert operation.

This helps avoid missing data and simplifies data entry by automatically filling common values.

Example

```
CREATE TABLE products (  
    name VARCHAR(100),  
    status VARCHAR(20) DEFAULT 'in_stock'  
);
```

Explanation

- If a value for status is not provided, it will automatically be set to 'in_stock'

Example

Insert without specifying status

```
INSERT INTO products (name)  
VALUES ('Laptop');
```

Stored result

- name → Laptop
- status → in_stock

CHECK Constraint

The **CHECK** constraint ensures that the values in a column satisfy a specified condition. It allows the database to enforce certain logical rules.

This is useful for enforcing business requirements such as minimum age limits, positive balances, or valid ranges.

Example

```
CREATE TABLE accounts (  
    id INT,  
    balance DECIMAL(10,2) CHECK (balance >= 0)  
);
```

Explanation

- The balance column cannot contain negative values
- Any attempt to insert a negative value will be rejected by the database

Example

Valid insert

```
INSERT INTO accounts VALUES (1, 500);
```

Invalid insert

```
INSERT INTO accounts VALUES (2, -100);
```

The second insert will fail because it violates the CHECK condition.

Naming Constraints

Constraints can also be given **explicit names** when they are defined. Naming constraints is helpful when managing or modifying them later.

Named constraints make it easier to identify and reference specific rules when using commands such as ALTER TABLE to modify or remove constraints.

Benefits of Naming Constraints

- Improves readability of the database schema
- Makes debugging easier when errors occur
- Helps when altering or dropping constraints later
- Provides clearer documentation of business rules

Example of Named Constraint

```
CREATE TABLE students (  
    roll_no INT PRIMARY KEY,  
    age INT CONSTRAINT chk_age CHECK (age >= 5),  
    email VARCHAR(100) UNIQUE
```

);

Explanation

- roll_no is the primary key of the table
- chk_age is the name of the CHECK constraint
- The constraint ensures that student age must be **at least 5 years**
- email must be unique across all records

Foreign Key

A **Foreign Key** is a constraint used to create a **relationship between two tables** in a relational database. It ensures that the value in one table corresponds to a valid value in another table.

Foreign keys help maintain **referential integrity**, which means that relationships between tables remain valid and consistent.

Typically, a foreign key in one table refers to the **primary key of another table**. This creates structured relationships such as **one-to-one**, **one-to-many**, or **many-to-many** relationships.

Creating a Database

Before creating tables and relationships, a database must be created and selected.

Example

```
CREATE DATABASE school;
```

```
USE school;
```

Explanation

- CREATE DATABASE creates a new database named **school**
- USE school selects the database so that all subsequent operations are performed within it

Creating Related Tables

To demonstrate foreign keys, two tables will be created:

- **classes** – stores information about classes
- **students** – stores information about students

Each student belongs to a class. This creates a **one-to-many relationship**, where:

- One class can have many students
- Each student belongs to one class

Creating the classes Table

The **classes** table stores information about different classes in the school.

```
CREATE TABLE classes (  
    class_id INT AUTO_INCREMENT PRIMARY KEY,  
    class_name VARCHAR(50) NOT NULL  
);
```

Explanation

- class_id is the **primary key** and uniquely identifies each class
- AUTO_INCREMENT automatically generates a new ID for every new class
- class_name stores the name of the class and cannot be NULL

Example records in the classes table might look like:

class_id	class_name
1	Mathematics
2	Science
3	History

Creating the students Table

The **students** table stores information about students and includes a foreign key that references the classes table.

```
CREATE TABLE students (  
    student_id INT AUTO_INCREMENT PRIMARY KEY,  
    student_name VARCHAR(100) NOT NULL,  
    class_id INT,  
    FOREIGN KEY (class_id) REFERENCES classes(class_id)  
    ON UPDATE CASCADE  
    ON DELETE SET NULL  
);
```

Explanation

- student_id uniquely identifies each student
- student_name stores the student's name and cannot be NULL
- class_id links each student to a class
- FOREIGN KEY (class_id) references the class_id column in the **classes** table

This relationship ensures that a student can only belong to an **existing class**.

Inserting Sample Data

Once the tables are created, sample data can be inserted to demonstrate the relationship.

Insert Data into classes Table

```
INSERT INTO classes (class_name)
```

```
VALUES
```

```
('Mathematics'),
```

```
('Science'),
```

```
('History');
```

Example data

class_id	class_name
1	Mathematics
2	Science
3	History

Insert Data into students Table

```
INSERT INTO students (student_name, class_id)
```

```
VALUES
```

```
('Alice', 1),
```

```
('Bob', 2),
```

```
('Charlie', 1);
```

Example data

student_id	student_name	class_id
1	Alice	1
2	Bob	2
3	Charlie	1

Explanation

- Alice and Charlie belong to **Mathematics** (class_id = 1)
- Bob belongs to **Science** (class_id = 2)

Understanding the Foreign Key Relationship

In the **students** table:

- **class_id** acts as a **foreign key**
- It references **class_id** in the **classes** table
- This ensures that a student cannot be assigned to a class that does not exist

For example, this insert would fail:

```
INSERT INTO students (student_name, class_id)
VALUES ('David', 10);
```

The operation fails because **class_id 10 does not exist in the classes table**.

Viewing Foreign Key Constraints

You can view the structure of the table and its constraints using the following command:

```
SHOW CREATE TABLE students;
```

This displays the SQL statement used to create the table, including the foreign key definition.

To see all foreign key relationships in the database, the **information_schema** table can be queried.

```
SELECT
    table_name,
    column_name,
    constraint_name,
    referenced_table_name,
    referenced_column_name
FROM
    information_schema.key_column_usage
WHERE
    referenced_table_name IS NOT NULL
    AND table_schema = 'school';
```

This query lists all foreign keys present in the **school database**.

Referential Actions in Foreign Keys

When defining foreign keys in MySQL, you can specify what should happen when the referenced data in the parent table is **updated or deleted**. These rules are called **referential actions**.

The two actions used in the example are:

- ON UPDATE CASCADE
- ON DELETE SET NULL

ON UPDATE CASCADE

Definition

If a value in the **parent table** (referenced column) is updated, the corresponding foreign key values in the **child table** are automatically updated.

Example

UPDATE classes

SET class_id = 10

WHERE class_id = 1;

Effect

All students who belonged to class **1** will now automatically have their class_id updated to **10**.

Example result

Before update

student_name	class_id
Alice	1
Charlie	1

After update

student_name	class_id
Alice	10
Charlie	10

This ensures that relationships remain valid after updates.

ON DELETE SET NULL

Definition

If a row in the **parent table** is deleted, the foreign key value in the **child table** is automatically set to **NULL**.

Example

DELETE FROM classes

WHERE class_id = 2;

Effect

Students who belonged to class **2** will no longer have a class assigned.

Example result

Before deletion

student_name	class_id
Bob	2

After deletion

student_name	class_id
Bob	NULL

This preserves the student record but removes the class relationship.

Why Use These Referential Actions

These options help manage relationships automatically and prevent inconsistent data.

Reasons to use them include:

- ON UPDATE CASCADE keeps foreign key relationships synchronized when the parent key changes
- ON DELETE SET NULL allows child records to remain while removing the relationship
- These rules reduce manual updates and prevent broken references

Alternative Referential Actions

MySQL also supports other actions for handling updates or deletions.

ON DELETE CASCADE

When a row in the parent table is deleted, all related rows in the child table are automatically deleted.

Example scenario

Deleting a class would delete all students belonging to that class.

ON DELETE RESTRICT

Prevents deletion of a parent row if related rows exist in the child table.

Example

If students belong to a class, that class cannot be deleted.

ON DELETE NO ACTION

In MySQL, this behaves the same as **RESTRICT**. The deletion is blocked if related rows exist.

ON DELETE SET DEFAULT

This option sets the foreign key to a default value when the parent row is deleted. However, **MySQL does not support this option**, although some other database systems do.

Joins

In relational databases, data is often stored in **multiple tables** to avoid redundancy and maintain proper structure. However, when retrieving information, we often need to combine data from these different tables. This is where **joins** are used.

A **join** allows data from two or more tables to be combined based on a related column between them. By joining tables, we can retrieve meaningful information that is spread across multiple tables.

For example, a database might store student information in one table and their marks in another table. A join can combine these tables to show each student's marks along with their name.

Example Tables

Consider two tables: **students** and **marks**.

Students Table

id	name
1	Alice
2	Bob

Marks Table

student_id	subject	score
1	Math	95
2	Math	88

2	Science	90
---	---------	----

In this example:

- students.id uniquely identifies each student
- marks.student_id refers to the student who received the mark

These columns allow the two tables to be connected using joins.

INNER JOIN

An **INNER JOIN** returns only the rows where there is a **matching value in both tables**. If a row in one table does not have a matching row in the other table, it will not appear in the result.

This type of join is used when we want only the records that exist in both tables.

Example

```
SELECT students.name, marks.subject, marks.score
FROM students
INNER JOIN marks ON students.id = marks.student_id;
```

Explanation

- The query combines the **students** and **marks** tables
- The condition students.id = marks.student_id defines how the tables are related
- Only rows where the IDs match in both tables will be returned

Result Example

name	subject	score
Alice	Math	95
Bob	Math	88
Bob	Science	90

If a student had no marks recorded, that student would **not appear** in the result.

LEFT JOIN (LEFT OUTER JOIN)

A **LEFT JOIN** returns **all rows from the left table** and the matching rows from the right table. If no match exists, the columns from the right table will contain **NULL values**.

This join is useful when we want to see **all records from the main table**, even if related data does not exist.

Example

```
SELECT students.name, marks.subject, marks.score
```

FROM students

LEFT JOIN marks ON students.id = marks.student_id;

Explanation

- The **students** table is the left table
- All students will appear in the result
- If a student has no marks, the subject and score fields will show **NULL**

Example Scenario

If a new student named **David** exists in the students table but has no marks yet, the result would look like:

name	subject	score
Alice	Math	95
Bob	Math	88
Bob	Science	90
David	NULL	NULL

This helps identify students who do not yet have marks recorded.

RIGHT JOIN (RIGHT OUTER JOIN)

A **RIGHT JOIN** returns **all rows from the right table** and matching rows from the left table. If no match exists, the columns from the left table will contain **NULL values**.

This join is less commonly used because the same result can usually be achieved using a LEFT JOIN by switching table order.

Example

SELECT students.name, marks.subject, marks.score

FROM students

RIGHT JOIN marks ON students.id = marks.student_id;

Explanation

- The **marks** table is the right table
- All rows from the marks table will appear
- If a mark exists without a matching student record, the student name will be NULL

Example Scenario

name	subject	score

Alice	Math	95
Bob	Math	88
Bob	Science	90
NULL	History	85

This could happen if a marks record was inserted with a student ID that does not exist in the students table.

CROSS JOIN OR FULL JOIN

A **CROSS JOIN** combines **every row from the first table with every row from the second table**. This produces all possible combinations of rows between the two tables.

Because the number of rows multiplies quickly, this join should be used carefully.

Example

```
SELECT students.name, marks.subject
```

```
FROM students
```

```
CROSS JOIN marks;
```

Explanation

If the **students table has 2 rows** and the **marks table has 3 rows**, the result will contain:

$2 \times 3 = 6$ rows

Example Result

name	subject
Alice	Math
Alice	Math
Alice	Science
Bob	Math
Bob	Math
Bob	Science

CROSS JOIN is useful in scenarios such as generating combinations, testing scenarios, or creating pairing datasets.

UNION in MySQL

The **UNION** operator in MySQL is used to **combine the results of two or more SELECT queries into a single result set**. It allows data retrieved from multiple tables or queries to be merged into one list.

UNION is useful when different tables store similar types of data and we want to display them together in one query result.

Why Use UNION

UNION is commonly used in situations where data is distributed across different tables but has a similar structure.

Common reasons to use UNION include:

- Combining similar data stored in different tables
- Merging results from multiple queries into one output
- Creating a consolidated list of records from multiple sources
- Comparing or aggregating similar datasets from different tables

Basic Example of UNION

Suppose we have two tables:

Customers Table

name	city
Alice	New York
Bob	Chicago

Vendors Table

name	city
David	Boston
Emma	Chicago

To combine the names and cities from both tables into one list, we can use UNION.

Example

```
SELECT name, city FROM customers
```

```
UNION
```

```
SELECT name, city FROM vendors;
```

Result

name	city
Alice	New York
Bob	Chicago
David	Boston
Emma	Chicago

The results from both queries are combined into a single output.

Rules for Using UNION

For the UNION operator to work correctly, certain conditions must be satisfied.

Same number of columns

All SELECT statements must return the **same number of columns**.

Example

```
SELECT name, city FROM customers
```

```
UNION
```

```
SELECT name, city FROM vendors;
```

Both queries return **two columns**, so the UNION works correctly.

Compatible data types

The corresponding columns must have **compatible or similar data types**.

For example:

- Text columns should be matched with text columns
- Numeric columns should be matched with numeric columns

Column matching by position

Columns are matched **by position**, not by column name.

Example

```
SELECT name, city FROM customers
```

```
UNION
```

```
SELECT vendor_name, vendor_city FROM vendors;
```

Even though the column names differ, the first column matches the first column and the second column matches the second column.

When UNION Does Not Work

Different number of columns

If the SELECT statements return a different number of columns, MySQL will produce an error.

Example

```
SELECT name, city FROM customers
```

```
UNION
```

```
SELECT name FROM vendors;
```

This fails because the first query returns **two columns**, while the second query returns **one column**.

Incompatible data types

If the data types are incompatible, MySQL may raise a type mismatch error or attempt implicit conversion.

Example

```
SELECT age FROM users
```

```
UNION
```

```
SELECT name FROM students;
```

This fails because age is numeric while name is a string.

UNION vs UNION ALL

By default, UNION removes duplicate rows from the result set. If duplicate values should be preserved, the **UNION ALL** operator is used.

UNION

- Combines results from multiple queries
- Automatically removes duplicate rows
- Slightly slower due to duplicate checking

Example

```
SELECT name FROM students
```

```
UNION
```

```
SELECT name FROM alumni;
```

If the same name appears in both tables, it will appear **only once** in the result.

UNION ALL

- Combines results from multiple queries
- Keeps duplicate rows
- Faster because it does not check for duplicates

Example

```
SELECT name FROM students
```

```
UNION ALL
```

```
SELECT name FROM alumni;
```

If a name appears in both tables, it will appear **twice** in the result.

Practical Example

Imagine two tables storing student data for different academic years.

students_2023

name	batch
Alice	2023
Bob	2023

students_2024

name	batch
Charlie	2024
David	2024

To create a combined list of students from both years:

```
SELECT name, batch FROM students_2023
```

```
UNION
```

```
SELECT name, batch FROM students_2024;
```

This produces a single list containing students from both tables.

Sorting the Combined Result

When using UNION, sorting must be applied **after the final SELECT statement** using the ORDER BY clause.

Example

```
SELECT name FROM students_2023
```

```
UNION
```

```
SELECT name FROM students_2024
```

```
ORDER BY name;
```

Explanation

- Results from both tables are combined first
- The final output is then sorted alphabetically by name

GROUP BY

The **GROUP BY** clause in MySQL is used to **group rows that share the same values** in specified columns. It is typically used alongside **aggregate functions** like COUNT(), SUM(), AVG(), MAX(), or MIN() to summarize data for each group.

Essentially, GROUP BY tells MySQL:

"Group these rows by the specified column(s), and then apply an aggregate function to each group."

Basic Examples

Count of Employees in Each Department

```
SELECT department, COUNT(*) AS total_employees  
FROM employees  
GROUP BY department;
```

- Groups employees by their **department**
- Counts how many employees are in each department

Output Example:

department	total_employees
IT	12
Sales	8
HR	5

Average Salary per Department

```
SELECT department, AVG(salary) AS avg_salary  
FROM employees
```

GROUP BY department;

- Groups employees by department
- Calculates **average salary** for each department

GROUP BY with Multiple Columns

You can group by more than one column to get **more detailed aggregations**.

Example: Count by department and job title

```
SELECT department, job_title, COUNT(*) AS count
```

```
FROM employees
```

```
GROUP BY department, job_title;
```

- Shows the number of employees for each **job title within each department**

The HAVING Clause

The **HAVING** clause filters **groups** after aggregation.

It is like WHERE, but it operates **after grouping**, based on aggregate results.

Example: Departments with more than 5 employees

```
SELECT department, COUNT(*) AS total_employees
```

```
FROM employees
```

```
GROUP BY department
```

```
HAVING COUNT(*) > 5;
```

- First, employees are grouped by department
- Then, only departments with **more than 5 employees** are displayed

Difference Between WHERE and HAVING

Clause	Used For	Example
WHERE	Filters rows before grouping	WHERE salary > 50000

HAVING	Filters groups after grouping	HAVING AVG(salary) > 60000
--------	--------------------------------------	----------------------------

Using Both Together:

```
SELECT department, AVG(salary) AS avg_salary  
FROM employees  
WHERE status = 'active'  
GROUP BY department  
HAVING AVG(salary) > 60000;
```

- WHERE filters only **active employees**
- GROUP BY groups them by department
- HAVING filters out departments with **low average salary**

WITH ROLLUP in MySQL

The **WITH ROLLUP** clause is used with GROUP BY to add **summary rows** (totals and subtotals) to the result set.

- Useful for generating **grand totals or subtotals** automatically
- Example:

```
SELECT department, SUM(salary) AS total_salary  
FROM employees  
GROUP BY department WITH ROLLUP;
```

Output Example:

department	total_salary
IT	120000
Sales	80000
HR	50000
NULL	250000

- The **NULL row** represents the **grand total** across all departments

MySQL Functions

String Functions

Function	Description	Example Usage
CONCAT()	Combine multiple strings	CONCAT('A','B') → 'AB'
LENGTH()	Returns length of a string in bytes	SELECT LENGTH('Hello'); → 5
CHAR_LENGTH()	Returns number of characters in a string	CHAR_LENGTH('हिंदी') → 5
LOWER()	Converts string to lowercase	LOWER('MySQL') → mysql
UPPER()	Converts string to uppercase	UPPER('hello') → HELLO
REPLACE()	Replaces part of a string with another value	REPLACE('abc','b','x') → axc
TRIM()	Removes leading and trailing spaces	TRIM(' hello ') → hello

Date and Time Functions

Function	Description	Example Usage
NOW()	Returns current date and time	NOW()
CURDATE()	Returns current date	CURDATE()
CURTIME()	Returns current time	CURTIME()
DATE()	Extracts date from datetime value	DATE(NOW())
MONTHNAME()	Returns the name of the month from a date	MONTHNAME('2025-05-03') → May
YEAR()	Extracts year from a date	YEAR(NOW())
DAY()	Returns day of the month	DAY('2025-05-03') → 3
DATEDIFF()	Returns number of days between two dates	DATEDIFF('2025-06-01','2025-05-01') → 31

Numeric Functions

Function	Description	Example Usage
ROUND()	Rounds a number to specified decimal places	ROUND(12.6789,2) → 12.68
FLOOR()	Rounds down to nearest whole number	FLOOR(5.9) → 5
CEIL()	Rounds up to nearest whole number	CEIL(5.1) → 6
ABS()	Returns absolute value	ABS(-10) → 10

MOD()	Returns remainder of division	MOD(10,3) → 1
RAND()	Generates random number between 0 and 1	RAND()

NULL Handling Functions

Function	Description	Example Usage
IFNULL()	Replaces NULL with a specified value	IFNULL(NULL,'N/A') → N/A
COALESCE()	Returns the first non-NULL value from a list	COALESCE(NULL,'"Hello') → "

Aggregate Functions

Function	Description	Example Usage
COUNT()	Counts number of rows	COUNT(*)
AVG()	Calculates average of numeric column	AVG(score)
SUM()	Calculates total sum of values	SUM(score)
MIN()	Returns smallest value	MIN(score)
MAX()	Returns largest value	MAX(score)

Views

A **View** in MySQL is a **virtual table** that is created using a SQL query. Unlike regular tables, a view does **not store data itself**. Instead, it displays data from one or more underlying tables whenever the view is queried.

A view works as a **saved query**. When you select data from a view, MySQL automatically runs the query defined in the view and returns the result as if it were a table.

Views can be used just like regular tables. You can perform operations such as **SELECT, filtering with WHERE, joining with other tables, and sorting** results.

Why Use Views

Views are useful for simplifying database operations and improving security and maintainability.

Common reasons for using views include:

- Simplifying complex queries by saving them with a meaningful name
- Hiding sensitive columns from certain users
- Showing only specific rows or columns from a table
- Reusing common query logic in applications or reports
- Improving readability and maintainability of SQL queries

Creating a View

To create a view, the **CREATE VIEW** statement is used along with a **SELECT** query.

Suppose there is an **employees** table containing many columns such as name, department, salary, address, phone number, and more. If only some of these columns should be visible, a view can be created to display just the required information.

Example

```
CREATE VIEW public_employees AS  
SELECT name, department, salary  
FROM employees;
```

This statement tells MySQL to create a view called **public_employees** that shows only the **name, department, and salary** columns from the employees table.

The view hides other columns that might exist in the original table.

Using a View

Once a view is created, it can be queried just like a normal table.

Example

```
SELECT * FROM public_employees;
```

This retrieves all rows from the view, displaying the columns defined in the view.

You can also apply filters or conditions when querying the view.

Example

```
SELECT * FROM public_employees  
WHERE department = 'IT';
```

This query returns only employees who belong to the **IT department**.

Updating or Replacing a View

If the structure of a view needs to be modified, it can be replaced using the **CREATE OR REPLACE VIEW** statement.

Example

```
CREATE OR REPLACE VIEW public_employees AS
```

```
SELECT name, department
```

```
FROM employees;
```

This replaces the existing view definition and removes the **salary column**, leaving only **name and department**.

Dropping a View

If a view is no longer needed, it can be removed from the database using the **DROP VIEW** command.

Example

```
DROP VIEW public_employees;
```

This deletes the view from the database but does **not affect the original table or its data**.

Important Notes About Views

There are several important characteristics of MySQL views.

- Views **do not store data**; they only display data from underlying tables
- If the data in the original table changes, the view automatically reflects those changes
- Views can simplify complex queries and improve code readability
- Not all views are **updatable**

Simple views created from a single table without grouping or joins usually allow operations such as **INSERT, UPDATE, or DELETE**. However, complex views that involve **joins, aggregation, or grouping** may not allow such modifications.

Example Use Case of Views

Imagine an application frequently runs the following query to find active customers with more than five orders.

```
SELECT customer_id, name, total_orders, status
```

```
FROM customers
```

```
WHERE status = 'active' AND total_orders > 5;
```

If this query is used multiple times in an application or report, repeating it each time can make the code harder to maintain.

Instead, a view can be created to store this query.

Example

```
CREATE VIEW top_customers AS  
  
SELECT customer_id, name, total_orders, status  
  
FROM customers  
  
WHERE status = 'active' AND total_orders > 5;
```

After creating the view, the application can simply query the view.

Example

```
SELECT * FROM top_customers;
```

Indexes

An **index** in MySQL is a special data structure used to **speed up data retrieval from a table**. Indexes allow the database engine to locate rows quickly without scanning the entire table.

Indexes are particularly useful when queries involve operations such as **WHERE filtering, JOIN conditions, ORDER BY sorting, or searching large datasets**.

A common analogy is the **index of a book**. Instead of reading every page to find a topic, you use the book's index to jump directly to the relevant pages. Similarly, MySQL uses indexes to quickly locate the rows that match a query condition.

Why Use Indexes

Indexes improve database performance, especially for large tables or frequently executed queries.

Main benefits of using indexes include:

- Faster data retrieval when searching or filtering records
- Improved performance for queries using **WHERE clauses**
- Faster joins between related tables
- Better performance when sorting results using **ORDER BY**
- Improved scalability as database size grows

Without indexes, MySQL often needs to perform a **full table scan**, which means checking every row to find matching data.

Creating an Index

Indexes are created using the **CREATE INDEX** statement. They can be created on one column or multiple columns depending on how queries are written.

Single Column Index

A **single column index** is created on one column of a table. It is useful when queries frequently search or filter based on that column.

Example

```
CREATE INDEX idx_email ON users(email);
```

This command tells MySQL to create an index named **idx_email** on the **email column** of the **users table**.

The database will now use this index to speed up queries that search for users by email.

Example query benefiting from this index

```
SELECT * FROM users  
WHERE email = 'user@example.com';
```

Multi-column (Composite) Index

A **composite index** is created on multiple columns. It is useful when queries frequently filter using more than one column.

Example

```
CREATE INDEX idx_name_city ON users(name, city);
```

This creates an index that includes both **name** and **city** columns.

This index works best when queries filter using these columns in the **same order**.

Example query

```
SELECT * FROM users  
WHERE name = 'Alice' AND city = 'Delhi';
```

The index allows MySQL to locate matching records more efficiently.

Dropping (Deleting) an Index

If an index is no longer needed, it can be removed using the **DROP INDEX** statement.

Example

```
DROP INDEX idx_email ON users;
```

This removes the index named **idx_email** from the **users table**.

Dropping unnecessary indexes can improve **write performance**, since MySQL no longer needs to maintain the index during inserts or updates.

When to Use Indexes

Indexes should be used strategically to improve database performance.

Good situations for using indexes include:

- Columns frequently used in **WHERE clauses**
- Columns used in **JOIN conditions**
- Columns used in **ORDER BY** or **GROUP BY**
- Unique identifiers such as **email, username, or ID**
- Large tables where queries regularly filter specific records

When Not to Use Indexes

Although indexes improve read performance, they also require extra storage and maintenance. Because of this, they should not be added unnecessarily.

Avoid indexes in the following cases:

- When the table is small, since MySQL can scan it quickly
- When a column is rarely used in searches or filters
- When a column contains very few unique values, such as a **gender column with only 'M' and 'F'**
- When the table experiences very frequent **INSERT, UPDATE, or DELETE operations**, because indexes must also be updated during these operations

Viewing Existing Indexes

To see all indexes defined on a table, MySQL provides the **SHOW INDEX** command.

Example

```
SHOW INDEX FROM users;
```

This command displays information about all indexes on the **users table**, including:

- Index name
- Indexed columns
- Index type
- Uniqueness of the index

Subqueries

A **subquery** in MySQL is a query **nested inside another SQL query**. Subqueries allow you to perform complex calculations, filtering, or temporary data transformations by breaking the logic into smaller, manageable steps. They can be used in **SELECT, FROM, or WHERE clauses** depending on the requirement.

Subqueries are enclosed in **parentheses** and return data that is used by the outer query. They can return:

- A **single value** (scalar)
- A **single row**
- A **full table**

Subquery in the WHERE Clause

Subqueries in the **WHERE clause** are commonly used for filtering results based on dynamic calculations.

Example: Employees who earn more than the average salary.

```
SELECT name, salary
```

```
FROM employees
```

```
WHERE salary > (
```

```
    SELECT AVG(salary)
```

```
    FROM employees
```

```
);
```

Explanation:

- First, the subquery calculates the **average salary** of all employees
- The outer query returns employees whose salary is **greater than the average**

Subquery in the FROM Clause

Subqueries in the **FROM clause** act as a **temporary table** that can be queried further.

Example: Departments with average salary above 50,000.

```
SELECT department, avg_salary
```

```
FROM (
```

```
    SELECT department, AVG(salary) AS avg_salary
```

```
    FROM employees
```

```
    GROUP BY department
```

```
) AS dept_avg
```

```
WHERE avg_salary > 50000;
```

Explanation:

- The inner query creates a temporary table of **department-wise average salaries**
- The outer query filters departments where the **average salary is above 50,000**

Subquery in the SELECT Clause

Subqueries in the **SELECT clause** allow you to calculate additional information for each row in the outer query.

Example: Count of projects per employee.

```
SELECT name,  
(SELECT COUNT(*)  
FROM projects  
WHERE projects.employee_id = employees.id) AS project_count  
FROM employees;
```

Explanation:

- For each employee, the subquery counts the number of projects they are assigned to
- This gives a result showing each employee and their **project count**

Correlated Subqueries

A **correlated subquery** depends on the outer query. It is executed **once for each row** of the outer query.

Example: Employees earning more than their department's average salary.

```
SELECT name, department, salary  
FROM employees e  
WHERE salary > (  
    SELECT AVG(salary)  
    FROM employees  
    WHERE department = e.department  
);
```

Explanation:

- For each employee, the subquery calculates the **average salary of their department**

- The outer query then returns employees whose salary is **greater than their department average**

Types of Subqueries

Type	Description
Scalar Subquery	Returns a single value
Row Subquery	Returns a single row with multiple columns
Table Subquery	Returns multiple rows and columns
Correlated Subquery	Refers to the outer query within the subquery

When to Use Subqueries

Subqueries are useful in scenarios such as:

- When the logic depends on **calculated values**, like averages or counts
- When filtering based on **dynamic conditions**
- When breaking down **complex queries** for readability and maintainability

When to Avoid Subqueries

- If the same result can be achieved using a **JOIN**, which is often faster
- When a **correlated subquery** executes repeatedly on a large table, which can be **performance-intensive**

Stored Procedures

A **Stored Procedure** is a **saved block of SQL code** that can be executed later by calling its name. It allows you to **group multiple SQL statements** into a single reusable unit, similar to a function in programming.

Stored procedures are useful for encapsulating **business logic**, improving performance, and reducing repetitive SQL code.

Why Use Stored Procedures

- Avoid repeating the same SQL logic in multiple queries or applications
- Improve performance by reducing **network traffic** (sending fewer queries from the client)
- Encapsulate complex business logic **inside the database**
- Enhance maintainability and readability of SQL code

Creating a Stored Procedure

When creating a stored procedure in MySQL, it is necessary to **temporarily change the statement delimiter** because a procedure contains multiple SQL statements, each ending with ;.

- **Why change the delimiter?**

MySQL ends a command at the first ;. By temporarily changing it (e.g., to // or \$\$), MySQL knows the procedure is not finished until it reaches the new delimiter.

Example: Simple Procedure to List All Employees

```
DELIMITER //
```

```
CREATE PROCEDURE list_employees()
```

```
BEGIN
```

```
    SELECT * FROM employees;
```

```
END //
```

```
DELIMITER ;
```

- This creates a procedure called **list_employees** that selects all rows from the **employees** table.

Calling a Stored Procedure

Use the CALL statement to execute a stored procedure.

```
CALL list_employees();
```

- Executes the list_employees procedure and returns all employees.

Stored Procedure with Parameters

Procedures can accept input values using the **IN** keyword.

Example: Get Employee Details by ID

```
DELIMITER //
```

```
CREATE PROCEDURE get_employee_by_id(IN emp_id INT)
```

```
BEGIN
```

```
    SELECT * FROM employees
```

```
    WHERE id = emp_id;
```

```
END //
```

```
DELIMITER ;
```

- Here, IN emp_id INT means the procedure takes an **integer input** called emp_id.

Calling the Procedure with a Parameter

```
CALL get_employee_by_id(3);
```

- Returns details of the employee with id = 3.

Dropping a Stored Procedure

To delete a procedure:

```
DROP PROCEDURE IF EXISTS get_employee_by_id;
```

- IF EXISTS ensures no error is thrown if the procedure does not exist.